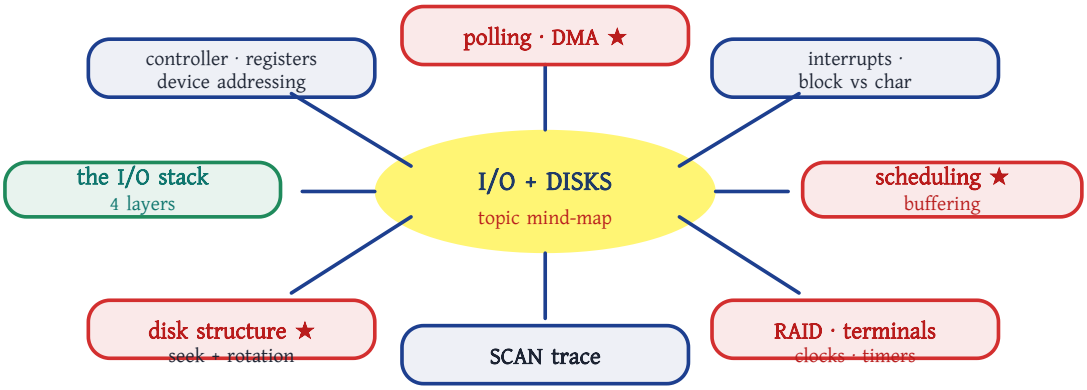
Input/Output, Disks & Clocks

controllers · interrupts · DMA · disk timings & scheduling · RAID · exam-ready in 11 sheets



↑ the whole topic on one page: red = high-yield, green = the layer the first questions come from

Syllabus coverage

The syllabus line is "**Input/Output: hardware, software, disk, terminals, clocks**". Memory management is in the Memory lesson, file allocation in the File Systems lesson.

High-yield list

① access time = seek + ½ rotation + transfer
② ½ rotation = **30 ÷ rpm** seconds
③ transfer time = bytes ÷ transfer rate
④ the CPU touches only the controller's **registers**
⑤ port-mapped = own IN/OUT space
⑥ DMA = **one** interrupt per block
⑦ SSTF starves · SCAN is fair · C-SCAN uniform
⑧ RAID 5 = (N − 1) disks, one may fail

Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

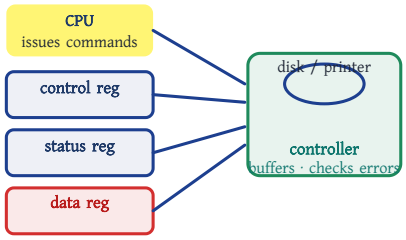
Contents

✓ I/O hardware & registers … p. 2
✓ Polling, interrupts, DMA … p. 3
✓ The I/O software stack … p. 4
✓ Scheduling & buffering … p. 5
✓ Disk structure & timings … p. 6
✓ Disk scheduling traces … p. 7
✓ Formatting, boot blocks, RAID … p. 8
✓ Terminals & clocks … p. 9 · Night p. 10 · Practice p. 11

What the whole lesson is for: Every I/O operation is one long wait, and the whole subject is the art of **keeping the CPU busy while a slow device is not ready.**

1 I/O Hardware: Devices, Controllers & Registers

Definition: I/O is any transfer of data between the computer and the outside world — a keyboard, a printer, a disk, a network. It is carried out by a **controller** (I/O subsystem, adapter or interface card), which does three jobs: it **moves data between the device and main memory**, it hides the device's speed from the CPU, and it lets the CPU **do other work while a transfer runs**.



the CPU's whole view of a device is **three registers** and an interrupt line

Piece	What it actually is
Device	the physical unit that does the real work: head, drum, print head, keys
Controller	the middleman — takes generic commands, buffers data, reports errors
Control register	what the device should do next (read, write, seek to cylinder 53)
Status register	how it is doing: **busy, ready, error** — the CPU reads this to decide
Data register	the one-word buffer where a byte crosses between CPU and device

Device addressing	Port-mapped (isolated) I/O	Memory-mapped I/O
Where the register lives	in a **separate port address space** — x86 gives 2^{16} = 64 K ports	at an **ordinary memory address** in the normal address space
Instructions used	special **IN / OUT** only — few addressing modes	ordinary **load / store**; every addressing mode works
What it costs	a separate instruction set; **no memory is wasted**	it eats memory address space, and those addresses must never be cached

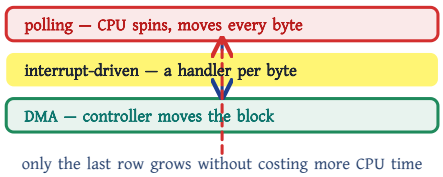
Interrupts: An **interrupt** lets a device call the CPU instead of the CPU calling the device. Three kinds: **trap** (an internal error — divide by zero, page fault), **interrupt** (a device, **asynchronous**), **software interrupt** (a system call). The CPU keeps each handler's address in a **vector table**; the **priority** decides who runs first.

Exam trap: (i) Memory-mapped I/O does **not** turn a device into memory — the address still lands on a controller register. (ii) It is **port-mapped** I/O that has the small instruction set; memory-mapped I/O is what **costs address space**. (iii) A trap is caused by the instruction being executed; an interrupt is **asynchronous**.

Memory trick: "**Three registers, one table.**" Control says *what*, status says *when*, data says *what byte*; the vector table says *who to jump to*. Everything else in this lesson is a way of avoiding the wait between control and status — and a modern machine still does exactly this through one PCIe/USB/SATA controller, **memory-mapped** registers and MSI-X interrupts.

2 Polling, Interrupt-Driven I/O & Direct Memory Access

Definition: The question of this sheet is always the same: **who carries the bytes, and who pays while it happens?** In **polling** the CPU carries them and pays the whole wait; in **interrupt-driven I/O** it carries them one byte at a time, paying a handler each time; in **DMA** the **controller** carries the whole block and the CPU pays **once, at the end.**



same 1 KB block, same device — only the CPU's share of the work changes

Method	Who moves data	Interrupts	Verdict
Polling	the CPU, word by word	none	CPU spins; only for slow, rare transfers
Interrupt-driven	the CPU, on every interrupt	**one per byte**	works otherwise; **worse than polling below ~1 MB/s**
DMA ★	the controller, the whole block	**one per block**	the only method a disk can use

How DMA shares the bus — and the two device classes:

Cycle stealing — one memory cycle at a time; the CPU slows but is **never blocked**

Transparent mode — the bus is used only when the CPU is **not** using it

Character device — a **byte stream in sequence**: no seek, nothing to schedule or cache

Burst mode — the controller holds the bus for the **whole block**; **best throughput**

Block device — fixed-size blocks, **random** access: buffering, caching and scheduling all pay off

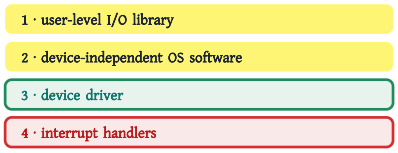
Worked example ✍ what does 1 KB really cost?: A device delivers **one byte every 1 µs** (1 MB/s); a block is **1 KB**; a handler costs **2 µs** of CPU. **(a)** The transfer takes 1024 µs. **(b) Polling:** the CPU is busy all 1024 µs moving bytes. **(c) Interrupt-driven:** 1024 × 2 µs = **2048 µs of CPU** — **twice** the transfer time, so at this rate the device is **faster with polling**. **(d) DMA:** 1 interrupt = 2 µs of CPU for 1024 bytes, i.e. **0.2 %** overhead.

Exam trap: (i) "Interrupt-driven is always better than polling" is **false**: a handler costs more CPU time than a few instructions. (ii) DMA still needs the CPU to **set up** the transfer — it removes the **copying**, not the CPU.

Memory trick: "**Slow ⇒ spin, fast ⇒ interrupt, faster ⇒ DMA.**" And "**Steal a cycle, hold a burst, wait quietly.**" — cycle stealing, burst and transparent in three phrases.

3 The Four Layers of I/O Software

Definition: I/O software is built in **four layers**, and the whole design is one question: **how far down does the hardware-specific knowledge go?** Only the layer directly above the controller knows the device exists, so a new disk needs one new driver — not one new program.



the stack, from the program that calls `read()` down to the wire

Layer	What it does
1 · User-level I/O software	the library the program calls; buffers output into lines, formats it, copies between user and kernel space
2 · Device-independent layer	uniform naming and protection, allocating and releasing the device, **buffering**, block sizing, error reporting
3 · Device driver ★	the **only** hardware-aware layer: turns a generic request into controller commands, starts the I/O, reads the status register
4 · Interrupt handlers	wait for the interrupt, save and restore context, identify the device, move the data, wake the waiting process

Exam trap: (i) The **device-independent** layer is where buffering and error handling live, **not** in the driver. (ii) A **driver** is kernel code; the **interrupt handler** is the **bottom** of the stack and runs in kernel mode on every interrupt.

Who answers what — and what happens without it:

Who names the device? layer 2 — a table maps the logical name to the physical one

Who knows the device? layer 3, the driver — and only layer 3

Who waits for the interrupt? layer 4, the handler

Who pads a 300-byte write into whole blocks? layer 2

In-kernel drivers — Linux — risk one bad driver taking the whole machine down. **User-mode drivers** isolate the damage: if the driver dies only that device is lost.

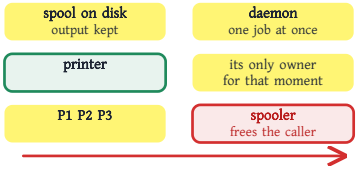
Memory trick: "**Library, common, driver, handler.**" Read the layers as "*who asked, what rules, how the hardware works, what woke us up*". A written question almost always wants these four in that order.

IT link: Loading a module for a new device — a USB printer, a graphics card — is exactly layer 3 being inserted at run time, and **nothing above it changes**.

4 I/O Scheduling, Buffering, Caching & Spooling

Definition: A device can serve only one request at a time, so they queue in a **device queue**; **I/O scheduling** is the policy that picks the next one, and it exists to stop a request being **starved**.

I/O scheduling policy	It serves...	Verdict
FCFS / fair-share	in arrival order, one round each per process	fair, but one long seek blocks everything behind it
Priority	the highest-priority request first	low priority can starve
SSTF (shortest seek first)	the request **nearest** the current position	**smallest average wait**, but starves far requests
EDF (earliest deadline first)	the request with the nearest deadline	the fairest policy; used for real-time service
Random (no-seek)	requests in any order	terrible on a disk; **optimal on an SSD**, where every block is equally far



spooling does not make the printer faster — it makes it **never idle**

Buffering — the three schemes and their cost:

Single: fill, then empty — the two sides take turns

Double: fill one while emptying the other

Circular: a ring; a used buffer is **reused**

Let C = time to compute, M = time to transfer one block

T = **C + M** unbuffered · **max(C,M) + M** single · **max(C,M)** double · **max(C, M+n) + M** with n circular buffers.

Caching vs spooling: **Caching** keeps a copy that can be fetched again cheaply, so it speeds up *reads*. **Spooling** stores a process's **whole output** on disk so an **exclusive** device can be shared. Caching **serves the same reader again**; spooling **serves many processes**.

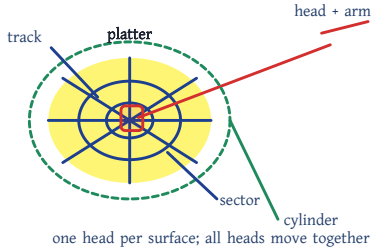
Worked example ✍ the printer case study: A laser printer finishes **1 page every 10 s** (6 pages/min). A program builds a 500-page report needing **0.2 s** of work per page. **(a)** Building the text = 500 × 0.2 = **100 s**; printing = 500 × 10 s = **5000 s ≈ 83.3 min**. **(b) No buffering:** 10 + 0.2 = 10.2 s per page. **Single:** max(0.2, 10) + 10 = 20 s per page ⇒ 500 pages = 10 000 s, **worse than no buffer at all**. **Double:** max(0.2, 10) = 10 s per page = 5000 s. **(c)** With **spooling** the program returns after 100 s and a daemon prints steadily while ten other programs queue their own output.

Exam trap: (i) **SSTF is I/O scheduling, SJF is CPU scheduling.** (ii) Double buffering **overlaps** compute and transfer. (iii) Caching serves the **same** data again; spooling serves **different** processes.

Memory trick: "**Buffer to bridge a gap, spool to share an exclusive device, cache to avoid a re-read.**"

5 Disk Structure and Access Time

Definition: A magnetic disk is a stack of **platters** on a **spindle**, turning at a fixed speed. Data lives on concentric **tracks**; each track is cut into fixed-length **sectors**, the smallest unit transferred; there is one **read/write head** per surface; and the tracks at the same radius on **every** surface form a **cylinder** — which can be read with no head movement at all.



a cylinder is not a shape — it is every track at the same radius

Term	Meaning
Platter / surface	one magnetised disc, one side usable ⇒ **heads = surfaces**
Track	a circle on one surface; **tracks per surface = cylinders**
Sector	**the unit of transfer — 512 B**, or **4 KiB** in the advanced format
Address	**CHS** = (cylinder, head, sector); modern disks expose **LBA**

Exam trap: (i) Old disks have the **same number of sectors on every track**; modern ones use **zone bit recording**, so outer tracks hold **more**. (ii) A cylinder is a **set of tracks**, not a shape.

The three timings and the one formula every question is built on:

Seek time T_s — move the head to the track

Rotational delay T_r = **½ × 60 ÷ rpm** s

Transfer time T_t = bytes ÷ transfer rate

Access time = $T_s + T_r + T_t$

Because **a sector arrives at a random point in a revolution, the average rotational delay is half of one revolution** — so T_r **= 30 ÷ rpm seconds**, and that is the shortest way to do the arithmetic.

Worked example ✍ average access time: A disk turns at **7200 rpm**, average seek is **4 ms**, its transfer rate is **200 MB/s**, and the read moves one **4 KiB** block. **(a)** One revolution = 60 ÷ 7200 = **8.33 ms**, so the average rotational delay is half of it = **4.17 ms**. **(b)** Transfer = 4096 B ÷ (200 × 10^6 B/s) = 2.05 × 10^{-5} s = **0.02 ms**. **(c)** Average access = 4 + 4.17 + 0.02 = **8.19 ms**. **(d)** Rotational delay is 4.17 ÷ 8.19 ≈ **51 %** of it — half of every access is spent merely *waiting for the sector*. **(e)** At **14400 rpm** that wait halves to 2.08 ms and the total falls to **6.10 ms**; doubling the transfer rate saves only 0.01 ms.

Memory trick: "**Seek, spin, ship.**" Head move, wait for the sector, move the bytes — always in that order, and **add them**. "**30 over rpm**" is the half-revolution wait in seconds.

In practice: The drive reorders requests internally and keeps a small cache, so what the OS sees is a mean latency closer to **8–10 ms** however far the head really moved.

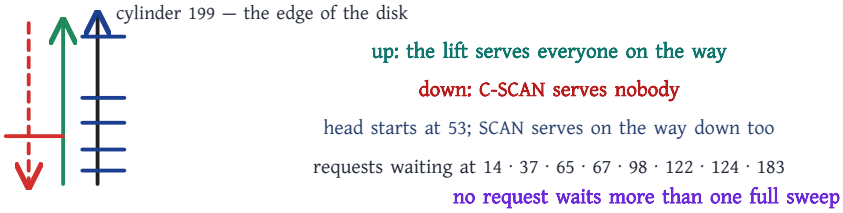
6 Disk Scheduling: FCFS, SSTF, SCAN, C-SCAN, LOOK

Definition: Requests queue up as **cylinder numbers**, and disk scheduling fixes the order in which the head visits them. The measure is the **total head movement** in cylinders, because every cylinder crossed costs a seek. **SSTF** minimises that total and starves; **SCAN** gives every request a **bounded wait** — that trade-off is what a written answer has to argue.

Algorithm	Order it serves in	Verdict
FCFS	arrival order	fair, no starvation, largest total movement
SSTF	whichever cylinder is **nearest** right now	**smallest total** — far requests may wait for ever
SCAN (elevator)	one way to the **physical end** of the disk, then reverse	the classic balanced answer
C-SCAN	one way only; on the return trip it **serves nobody**	**most uniform wait** — like a queue, not a lift
LOOK / C-LOOK	SCAN, but stopping at the **last request** instead of at the edge	the practical versions; far less wasted travel

Worked trace ✍️ head at cylinder 53, queue in arrival order 98, 183, 37, 122, 14, 124, 65, 67; disk 0–199:

Algorithm	Head movement, step by step	Total
FCFS	53→98→183→37→122→14→124→65→67 : 45+85+146+85+108+110+59+2	**640**
SSTF	53→65→67→37→14→98→122→124→183 : 12+2+30+23+84+24+2+59	**236**
SCAN ↑	53→65→67→98→122→124→183→199, then down to 37→14 : 130+16+162+23	**331**
LOOK ↑	as SCAN, but reversing at 183 and not 199 : 130+169	**299**
C-SCAN ↑	up to 199 (146), jump to 0 (199), then 0→14→37 (14+23)	**382**
C-LOOK ↑	up to 183 (130), jump back to 14 (169), then up to 124 (110)	**409**



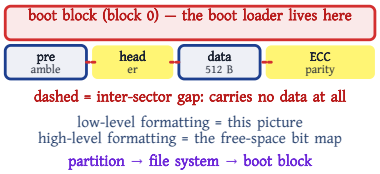
the elevator family: every request is reached within one full sweep

Exam trap: (i) **SCAN** reaches the **end of the disk**; **LOOK** stops at the last request — that is the whole 331 − 32 = **299**. (ii) **C-SCAN** services in one direction only. (iii) "Smallest total movement" and "smallest average wait" are **not** the same question.

Memory trick: **"SSTF is nearest, SCAN is the lift, C-SCAN is the queue."** A lift takes people both ways up and down; a queue is served in one direction and the return trip carries nobody.

7 Formatting, Boot Blocks, RAID & Solid-State Disks

Definition: A blank disk must be marked before anything can be stored on it, at **two levels**. **Low-level (physical) formatting** divides the surface into sectors and writes the **preamble, header, ECC and inter-sector gaps**; it is the factory's job. **High-level (logical) formatting** writes the **free-space structure** — the bit map marking every block.



why a formatted disk always shows less than its nominal capacity

From a blank disk to a bootable system:

1 · Low-level format: sectors, gaps, ECC

2 · Partition table: MBR (4 primary, 2 TB) or GPT (128, huge)

3 · High-level format: a file system on one partition

4 · Boot block: block 0 holds the boot loader

The BIOS/UEFI reads block 0 and runs the boot loader, which loads the kernel. **Bad blocks** are handled by **sparing**: the content moves to a reserved sector and the number is remapped.

Exam trap: (i) Sectors are separated by **inter-sector gaps**, so **a formatted disk never shows its full nominal capacity**. (ii) Low-level formatting is the **factory's** job; a bad block is **spared**.

RAID level	How it works	Usable capacity (min. disks)	Disks that may fail
0 — striping	data split across all disks, no redundancy	**100 %** = N × size (2)	0 — lose one, lose all
1 — mirroring	an exact copy on every disk	one disk's size (2)	N − 1
5 — distributed parity ★	parity spread round-robin	**(N − 1) × size** (3)	**1**
6 — double parity	RAID 5 with **two** parity blocks	**(N − 2) × size** (4)	**2**
10 — 1 + 0	mirror pairs, then stripe across them	50 % of raw (4)	one per pair

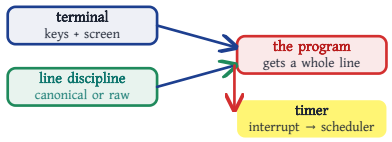
Worked example ✍ RAID arithmetic: **(a)** RAID 0 of 4 × 1 TB → **4 TB**, no redundancy. **(b)** RAID 1 of 4 × 1 TB → **1 TB**, any **3** disks may fail. **(c)** RAID 5 of 4 × 1 TB → (4 − 1) = **3 TB**, one disk tolerated, but rebuilding rewrites everything. **(d)** RAID 6 of 5 × 1 TB → (5 − 2) = **3 TB** with **2** tolerated; RAID 10 of 8 × 1 TB → 8 ÷ 2 = **4 TB**. **(e)** A **small** write to RAID 5 costs **4 disk I/Os** — read old data, read old parity, write new data, write new parity.

Memory trick: "**Zero stripes, one mirror, five one parity, six two, ten half.**" Mirror-then-stripe is written **10**; stripe-then-mirror is **01** — the digits are the order of the operations.

SSD vs magnetic disk: No head, no rotation, **random access ≈ 0.1 ms** against 4–10 ms; symmetric reads and writes; **wear levelling** spreads writes over the cells; TRIM marks dead blocks. Still dearer per TB.

8 Terminals, Line Discipline, Clocks & Timer Interrupts

Definition: A **terminal** is the device through which a person talks to the computer — a keyboard for input, a display for output, **character by character**. A **line discipline** in the device-independent layer sits between them and decides buffering, echo and special keys.



three keystrokes are three interrupts; one `read()` is one line

Terminal	What it is
Character terminal	keyboard + character display; a **byte stream** in, text out — a **character device**
Graphics terminal	accepts **escape sequences** and draws at the pixel level
Line discipline	**canonical** assembles a whole line and echoes it; **raw** passes each keystroke at once
Pseudo-terminal	what SSH uses: a file the terminal writes and the line discipline reads, then sends over the network

Exam trap: A terminal is **not** a "remote console": it is an I/O device, so it has a **driver**, a **line discipline** and **interrupt-driven input**.

The three jobs of a clock — and what breaks without each one:

1 · **Measure elapsed time.** A free-running counter read on request

2 · **Enforce time limits. the interval timer** — what makes scheduling **preemptive**

3 · **Account CPU time.** charge each process for the ticks it used

Time-of-day clock: keeps wall-clock time — the battery-backed CMOS/RTC

Hardware is simple: a counter is decremented every clock tick and, on reaching zero, it **raises an interrupt** and is reloaded. That single interrupt is what stops a runaway process from keeping the CPU for ever. A typical tick is **10 ms**, so eight round-robin processes each get 10 ms in an 80 ms round.

Clock vs timer: A **clock** keeps the **time**; a **timer counts** and **raises an interrupt**. A time-of-day clock can be read but never interrupts; an interval timer interrupts on schedule. Real hardware merges them into one counter with a compare register.

Memory trick: "**The clock tells the time; the timer tells the process to stop.**" Only an interrupt can move a running process off the CPU, and that interrupt always comes from a timer.

Exam trap: (i) Without a timer interrupt, scheduling is **non-preemptive**. (ii) The time-of-day clock does **not** enforce the quantum; Linux now runs **tickless**.

Night-Before-the-Exam Sheet

Fact sheet:

Controller = the middleman

Control, status, data registers

Status says **busy / ready / error**

Vector table = handler addresses

Port-mapped = IN/OUT, own space

Memory-mapped = load/store, **eats space**

Trap = internal error

Interrupt = device, asynchronous

Polling = CPU spins

Interrupt = CPU per byte

DMA = controller, whole block

DMA = **1** interrupt per block

Steal a cycle · hold a burst · wait quietly

Block device = blocks · char = stream

I/O stack: library, common, driver, handler

Only the driver knows the device

I/O policies: FCFS, priority, SSTF, EDF

On an SSD, **random is optimal**

Buffer = speed + size mismatch

Cache = avoid a re-read

Spool = share an exclusive device

Spooling uses a **daemon**

T = max(C,M) with double buffering

Platters → tracks → sectors → cylinder

T_r = 30 ÷ rpm seconds

T_t = bytes ÷ rate

Access = seek + spin + ship

Same sectors per track: **no** (ZBR)

SSTF starves · SCAN is the lift

C-SCAN = a queue, one direction

LOOK stops at the last request

Formatting: gaps · ECC · sparing

Boot block = block 0 = the loader

RAID 0/1/5/6/10: 0 · 1 · N−1 · N−2 · N/2

Small RAID 5 write = **4** I/Os

SSD ≈ 0.1 ms · disk ≈ 8 ms

Line discipline: canonical or raw

Clock keeps time · timer interrupts

Top 12 one-liners for MCQs

1. The CPU sees a device as **three registers**
2. Memory-mapped I/O costs **address space**
3. Interrupt-driven I/O is **1 interrupt per byte**
4. DMA interrupts the CPU **once per block**
5. 7200 rpm ⇒ average rotational delay **4.17 ms**
6. 7200 rpm + 4 ms seek + 4 KiB ⇒ **8.19 ms**
7. A cylinder = every track at the same radius
8. This queue under SSTF from 53 ⇒ **236**
9. The same queue under SCAN ⇒ **331**; LOOK ⇒ **299**
10. RAID 5 of 4 disks ⇒ **3** usable, **1** may fail
11. Keyboard, mouse, printer = **character** devices
12. Only a **timer interrupt** can preempt a process

Top traps: saying polling is efficient because it needs no interrupt logic · saying interrupt-driven I/O always beats polling · DMA said to remove the CPU instead of the copying · a sector said to be the same size on every track · a cylinder drawn as a shape · SCAN said to stop at the last request · C-SCAN said to serve on the return trip · **RAID 5 said to survive two failures** · RAID 10 written 01 · low-level formatting said to be the user's job · the time-of-day clock said to enforce the quantum.

All tricks together: "Three registers, one table" · "Slow ⇒ spin, fast ⇒ interrupt, faster ⇒ DMA" · "Steal a cycle, hold a burst, wait quietly" · "Library, common, driver, handler" · "Buffer to bridge, spool to share, cache to avoid a re-read" · "Seek, spin, ship" · "30 over rpm" · "SSTF is nearest, SCAN is the lift, C-SCAN is the queue" · "Zero stripes, one mirror, five one, six two, ten half" · "The clock tells the time; the timer tells the process to stop".

✓ Practice MCQs (exam pattern, 10 marks)

1. A controller transfers a whole block straight to memory and interrupts the CPU only at the end. That is:
(a) programmed I/O (b) polling
(c) direct memory access (d) spooling

2. The main cost of memory-mapped I/O over port-mapped I/O is that it:
(a) needs a special instruction (b) cannot raise interrupts
(c) must use DMA (d) consumes memory address space

3. At a low data rate a handler may cost more than the transfer, so which method can be **faster**?
(a) DMA (b) polling
(c) cycle stealing (d) spooling

4. The tracks at the same radius on every platter surface form a:
(a) cylinder (b) sector
(c) surface (d) spindle

5. A disk running at 7200 rpm has an average rotational latency of:
(a) 8.33 ms (b) 2.08 ms
(c) 4.17 ms (d) 16.7 ms

6. Which of these is a **character**, not a block, device?
(a) hard disk (b) SSD
(c) USB flash drive (d) keyboard

7. Which disk policy guarantees no request waits more than one full sweep?
(a) FCFS (b) SCAN
(c) SSTF (d) random

8. Head at 53, queue 98, 183, 37, 122, 14, 124, 65, 67. Total movement under **SSTF**:
(a) 236 (b) 640
(c) 331 (d) 299

9. Moving the head to the required track is the:
(a) rotational latency (b) transfer time
(c) queueing delay (d) seek time

10. What does a time-of-day clock do that an interval timer does not?
(a) interrupt on schedule (b) keep the wall-clock time
(c) end a time slice (d) charge CPU time to a process

Answer key → 1-c · 2-d · 3-b · 4-a · 5-c · 6-d · 7-b · 8-a · 9-d · 10-b

Written question: Explain the layers of I/O software. Describe DMA and its three modes of bus access. 10

1. **The four layers** — (1) **user-level I/O library**: buffers output into lines. (2) **Device-independent OS software**: uniform naming, allocating the device, buffering, block sizing, error reporting. (3) **Device driver**: the only hardware-aware layer. (4) **Interrupt handlers**: run on the interrupt, move the data, wake the waiting process. 4
2. **DMA** — the controller itself transfers a **whole block** between the device and main memory; the CPU only sets up the memory address, the count and the device port, then continues other work and is **interrupted once when the block has moved.** 3
3. **Three modes** — **cycle stealing**: one memory cycle at a time, the bus returned, so the CPU slows but is never blocked. **Burst mode**: the bus is held for the whole block. **Transparent mode**: the bus is used only when the CPU is idle. 3

Before the exam ✓

- ✓ Can name the four layers of the I/O stack
- ✓ Can tell polling, interrupt and DMA apart by CPU cost
- ✓ Can compute average access time from rpm, seek and rate
- ✓ Can trace a request queue under SCAN and total the movement
- ✓ Can do RAID capacity arithmetic and say how many disks may fail
- ✓ Can separate the clock from the timer